| DS | **Session** | Graph | **Question Information** | Level 1 Challenge 85 |
|---|---|---|---|---|

**Question description**

You have an undirected graph consisting of $n$ vertices with weighted edges.

A simple cycle is a cycle of the graph without repeated vertices. Let the *weight* of the cycle be the XOR of weights of edges it consists of.

Let's say the graph is *good* if all its *simple* cycles have weight 1. A graph is bad if it's not good.

Initially, the graph is empty. Then $q$ queries follow. Each query has the next type:

- $u$ $v$ $x$ — add edge between vertices $u$ and $v$ of weight $x$ if it doesn't make the graph bad.

For each query print, was the edge added or not.

**Constraints**

3≤$n$≤3·10^5; 1≤$q$≤5·10^5

1≤$u$,$v$≤$n$; $u$≠$v$; 0≤$x$≤1

**Input**

The first line contains two integers $n$ and $q$ — the number of vertices and queries.

Next $q$ lines contain queries — one per line. Each query contains three integers $u$, $v$ and $x$ — the vertices of the edge and its weight.

**It's guaranteed that there are no multiple edges in the input.**

**Output**

For each query, print YES if the edge was added to the graph, or NO otherwise (both case-insensitive).

## Logical Test Cases

### Test Case 1

INPUT (STDIN)

```
9 12
6 1 0
1 3 1
3 6 0
6 2 0
6 4 1
3 4 1
2 4 0
2 5 0
4 5 0
7 8 1
8 9 1
9 7 0
```

EXPECTED OUTPUT

```
YES
YES
YES
YES
YES
NO
YES
YES
NO
YES
YES
NO
```

### Test Case 2

INPUT (STDIN)

```
9 6
1 3 1
3 6 0
6 4 1
2 4 0
4 5 0
7 8 1
```

EXPECTED OUTPUT

```
YES
YES
YES
YES
YES
YES
```

## Mandatory Test Cases

### Test Case 1

KEYWORD

```
int dfs1(int np,int lst)
```

## Complexity Test Cases

### Test Case 1

CYCLOMATIC COMPLEXITY

12

### Test Case 2

TOKEN COUNT

1255

### Test Case 3

NLOC

78

# Code

C good_graph_cycles.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAXN 300005

int parent[MAXN];
int parity[MAXN];

int find_set(int v) {
    if (v == parent[v]) return v;
    int root = find_set(parent[v]);
    parity[v] ^= parity[parent[v]];
    return parent[v] = root;
}

int dfsi(int np, int lst) {
    return np ^ lst;
}

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;

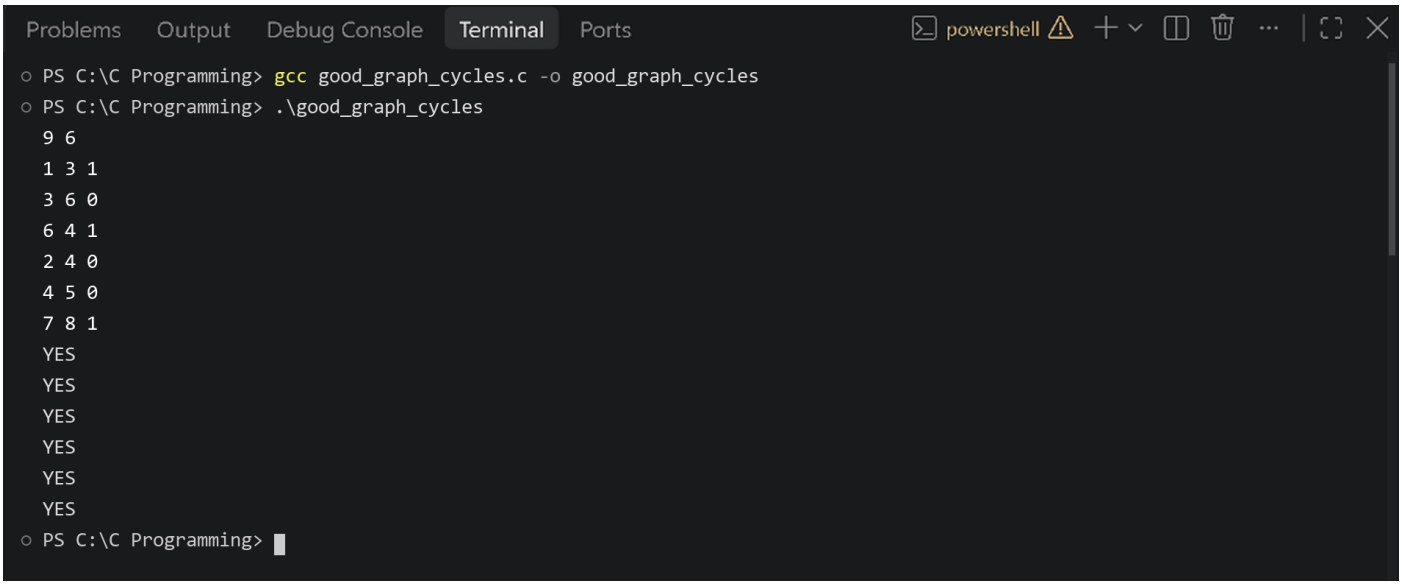
    for (int i = 1; i <= n; i++) {
        parent[i] = i;
        parity[i] = 0;
    }

    while (q--) {
        int u, v, x;
        scanf("%d %d %d", &u, &v, &x);
        int ru = find_set(u);
        int rv = find_set(v);

        if (ru != rv) {
            parent[ru] = rv;
            parity[ru] = parity[u] ^ parity[v] ^ x;
            printf("YES\n");
        } else {

            if ((parity[u] ^ parity[v] ^ x) == 1) {
                printf("YES\n");
            } else {
                printf("NO\n");
            }
        }
    }
    dfsi(0, 0);
    return 0;
}
```

## Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc good_graph_cycles.c -o good_graph_cycles
PS C:\C Programming> .\good_graph_cycles
9 6
1 3 1
3 6 0
6 4 1
2 4 0
4 5 0
7 8 1
YES
YES
YES
YES
YES
YES
PS C:\C Programming>